

Workshop LLM

Développement d'applications basées sur les LLMs

Exercice : Agent avec outils et routage LangGraph

Pr. Youssef Saadi

Faculté des Sciences et Techniques de Béni Mellal
Université Sultan Moulay Slimane — ICC

1. Contexte et objectifs

1.1 Contexte

Dans cet exercice de workshop, vous allez construire un **assistant académique intelligent** capable de répondre à trois types de requêtes : des questions de cours, des calculs mathématiques simples, et des demandes de définitions de termes techniques. L'agent doit détecter automatiquement le type de requête et router vers l'outil approprié, sans intervention de l'utilisateur.

2. Énoncé de l'exercice

2.1 Installation

Commencez par installer les dépendances requises :

```
pip install langgraph langchain-core langchain-groq
```

2.2 Partie 1 — Définir les outils

Créez un fichier `tools.py` et définissez les trois outils suivants en utilisant le décorateur `@tool` de LangChain.

Outil 1 : `tool_cours`

Signature : `tool_cours(sujet: str) -> str`

Cet outil doit :

- Recevoir un sujet sous forme de chaîne de caractères
- Retourner une explication pédagogique du concept demandé
- Pour l'exercice, utiliser un dictionnaire local avec quelques entrées prédéfinies (rétropropagation, gradient descent...)
- Retourner un message générique si le sujet n'est pas trouvé

Outil 2 : `tool_calcul`

Signature : `tool_calcul(expression: str) -> str`

Cet outil doit :

- Recevoir une expression mathématique sous forme de chaîne (ex : "144 / 12 + 7 * 3")
- Valider que l'expression ne contient que des caractères autorisés (chiffres et opérateurs arithmétiques)
- Évaluer l'expression et retourner le résultat formaté
- Gérer les exceptions et retourner un message d'erreur explicite

Outil 3 : tool_definition

Signature : `tool_definition(terme: str) -> str`

Cet outil doit :

- Recevoir un terme technique (IA/ML) sous forme de chaîne
- Retourner une définition courte depuis un glossaire local
- Couvrir au minimum : overfitting, embedding, attention, RAG
- Retourner un message adapté si le terme est absent du glossaire

Important

Le décorateur `@tool` utilise la docstring de la fonction pour décrire l'outil au LLM. Une docstring précise améliore significativement la qualité du routage automatique. Exemple : `"""Explique un concept de cours d'IA ou d'informatique."""`

2.3 Partie 2 — Construire le graph LangGraph

Créez un fichier `graph.py` et implémentez les éléments suivants.

Étape A : définir AgentState

Définissez un `TypedDict` nommé `AgentState` contenant un seul champ :

Étape B : créer le nœud agent

Définissez une fonction `node_agent(state: AgentState) -> dict :`

Étape C : écrire le routeur conditionnel

Définissez une fonction `should_continue(state: AgentState) -> str :`

Étape D : assembler et compiler le graph

Construisez le `StateGraph` en ajoutant :

1. Les nœuds : agent et tools (en utilisant `ToolNode` de `langgraph.prebuilt`)
2. Le point d'entrée : `set_entry_point("agent")`
3. L'arête conditionnelle depuis agent via `add_conditional_edges`
4. L'arête fixe de tools → agent (le cycle de réflexion)
5. La compilation : `app = graph.compile()`

Concept clé — le cycle agentique

L'arête `tools → agent` est la clé du cycle agentique. Après l'exécution d'un outil, le résultat est ajouté aux messages et le LLM est rappelé pour formuler une réponse finale en langage naturel, en intégrant le résultat de l'outil.

2.4 Partie 3 — Tester l'agent

Créez un fichier main.py et testez l'agent avec les trois requêtes suivantes. Pour chaque requête, identifiez quel outil a été sélectionné et pourquoi.

```
# Requête 1 : outil cours
"Explique-moi ce qu'est la rétropropagation"

# Requête 2 : outil calcul
"Calcule  $144 / 12 + 7 * 3$ "

# Requête 3 : outil définition
"Donne-moi la définition de l'overfitting"
```